

M08 / PROTECTED ANSWERS

M08 Module Revision Test Answers

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	5
How to use the protected answers	5
A01 - DE08-01 Ingestion architecture: source -> landing -> staging	5
Expected answer / validation	5
A02 - DE08-02 Defensive CSV ingestion	5
Expected answer / validation	5
A03 - DE08-03 JSON and nested-record ingestion	5
Expected answer / validation	5
A04 - DE08-04 Parquet ingestion and metadata inspection	6
Expected answer / validation	6
A05 - DE08-05 REST API contracts, status codes and timeouts	6
Expected answer / validation	6
A06 - DE08-06 API authentication, pagination and rate limits	6
Expected answer / validation	6
A07 - DE08-07 Retries, backoff, jitter and transient failures	6
Expected answer / validation	6
A08 - DE08-08 Database extraction and safe query boundaries	6
Expected answer / validation	6
A09 - DE08-09 Historical batch landing and raw metadata	7
Expected answer / validation	7
A10 - DE08-10 Incremental loads and high-water marks	7

Expected answer / validation	7
A11 - DE08-11 CDC concepts and change-event semantics	7
Expected answer / validation	7
A12 - DE08-12 Schema drift, compatibility, time zones and quarantine	7
Expected answer / validation	7
A13 - DE08-13 Checkpoint/state management and replay safety	7
Expected answer / validation	7
A14 - DE08-14 Idempotent landing and content identity	8
Expected answer / validation	8
A15 - DE08-15 Late-arriving data, event time and processing time	8
Expected answer / validation	8
A16 - DE08-16 Ingestion observability: counts, latency and run metadata	8
Expected answer / validation	8
A17 - DE08-17 Ingestion integration, reconciliation and backfill proof	8
Expected answer / validation	8
M08 WORKED VALIDATION APPENDIX	8
A01 - DE08-01 Ingestion architecture	8
A02 - DE08-02 Defensive CSV ingestion	9
A03 - DE08-03 JSON and nested-record ingestion	9
A04 - DE08-04 Parquet ingestion	10
A05 - DE08-05 REST contracts/status/timeouts	10
A06 - DE08-06 Auth, pagination and rate limits	11
A07 - DE08-07 Retries/backoff/jitter	11

A08 - DE08-08 | Database extraction 11

A09 - DE08-09 | Historical raw landing 12

A10 - DE08-10 | Incremental loads/high-water marks 12

A11 - DE08-11 | CDC semantics 13

A12 - DE08-12 | Schema drift/time zones/quarantine 13

A13 - DE08-13 | Checkpoint/state and replay 13

A14 - DE08-14 | Idempotent landing/content identity 14

A15 - DE08-15 | Late data/event vs processing time 14

A16 - DE08-16 | Ingestion observability 15

A17 - DE08-17 | Integration/backfill proof 15

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

PROTECTED TEST ANSWERS

Use this file to validate and repair, not to erase your first attempt. Compare the expected control with what you actually observed, close this guide, then solve a changed retry if the competency was weak.

Coverage: 17 canonical lessons

Route: learner book -> practice -> revision -> closed-book test -> protected answers -> changed retry.

How to use the protected answers

Open only the lesson you attempted. The guide gives the expected control/behavior, not permission to copy. If you used it, close it and perform a changed retry before marking the lesson mastered.

A01 - DE08-01 | Ingestion architecture: source -> landing -> staging

Expected answer / validation

Strong evidence separates immutable raw landing from rebuildable staging, keeps quarantine with reasons, and commits state only after complete success. A retry creates new run evidence rather than overwriting yesterday.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A02 - DE08-02 | Defensive CSV ingestion

Expected answer / validation

Good CSV: 4 source rows -> 4 accepted, 0 quarantine. Bad fixture must preserve raw bytes and classify defects; accepted+quarantine must equal source rows. Leading-zero IDs stay text.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A03 - DE08-03 | JSON and nested-record ingestion

Expected answer / validation

J3001 produces exactly two item-grain rows. State the key (order_id,line_no) for this fixture and reconcile item totals from source versus flattened rows.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A04 - DE08-04 | Parquet ingestion and metadata inspection

Expected answer / validation

Parquet logical row count and keys match source, schema is inspected, projected read contains only requested columns, and metadata row-group evidence is saved. Native pyarrow execution is required on learner PC.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A05 - DE08-05 | REST API contracts, status codes and timeouts

Expected answer / validation

/health=200; unauthenticated order endpoint=401; authenticated valid call=200; missing path=404. JSON decode is never used as proof of HTTP success and timeout is explicit.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A06 - DE08-06 | API authentication, pagination and rate limits

Expected answer / validation

page_size=3 yields all 10 unique v1 IDs; one page-2 429 is retried according to Retry-After; 401 is not repeatedly retried; token is absent from saved logs.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A07 - DE08-07 | Retries, backoff, jitter and transient failures

Expected answer / validation

One-time 500 recovers under bounded retry. 429 may retry. Wrong auth fails quickly. Attempt count and backoff are visible. Write retries are not assumed safe without idempotency semantics.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A08 - DE08-08 | Database extraction and safe query boundaries

Expected answer / validation

After (08:15,D5002) the local extract returns D5003 then D5004. Query uses explicit columns, parameters and deterministic order. Read-only connection rejects mutation.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A09 - DE08-09 | Historical batch landing and raw metadata

Expected answer / validation

Both historical files land in one named run, each manifest entry records original name/bytes/SHA256, and a second run does not overwrite first-run evidence.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A10 - DE08-10 | Incremental loads and high-water marks

Expected answer / validation

Initial v1 ingest returns 10 unique records and final state (2026-08-05T09:20:00Z,A4010). A4007/A4008 tie proves timestamp-only watermarks unsafe. No-new-data rerun returns zero.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A11 - DE08-11 | CDC concepts and change-event semantics

Expected answer / validation

Applying ordered CDC events leaves A5001 status paid and removes A5002 under current-state semantics while preserving event history. Duplicate event ID does not duplicate effect.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A12 - DE08-12 | Schema drift, compatibility, time zones and quarantine

Expected answer / validation

v2 raw page lands but contributes zero v1 staging rows; records are quarantined as unsupported/incompatible and live v1 checkpoint does not advance. Offset-less time is not silently assigned local time.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A13 - DE08-13 | Checkpoint/state management and replay safety

Expected answer / validation

Replay creates new landing evidence but live checkpoint remains byte-identical under no-commit policy; failure before commit leaves prior state intact; checkpoint ownership is explicit.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A14 - DE08-14 | Idempotent landing and content identity

Expected answer / validation

Two landings can keep identical bytes/run evidence while logical staging remains one row per business key. SHA proves byte identity, not business identity.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A15 - DE08-15 | Late-arriving data, event time and processing time

Expected answer / validation

L2 is late under 60-minute policy; event time is preserved. Classification changes predictably under a new tolerance and does not rewrite the business timestamp.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A16 - DE08-16 | Ingestion observability: counts, latency and run metadata

Expected answer / validation

Run metadata includes counts/retries/state/status and accepted+quarantined reconciles to source scope. Missing row with SUCCESS is detected by equation, not hidden by logs.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A17 - DE08-17 | Ingestion integration, reconciliation and backfill proof

Expected answer / validation

Integration proof includes immutable landing, contract validation, quarantine, bounded retry, composite state, replay/idempotency, late/event-time reasoning, structured run metadata, failure injection and handoff.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

M08 | WORKED VALIDATION APPENDIX

Use this appendix only after a saved attempt. It turns short validation points into concrete solved examples, failure checks and changed retries.

A01 - DE08-01 | Ingestion architecture

Concrete solved example

Source file lands immutably under a run/object identity; staging is rebuilt from that evidence; invalid rows go to quarantine with reasons; external progress is committed only after durable

target validation.

Worked proof / evidence

source -> raw/run_123/hash -> staging -> accepted + quarantine -> target -> commit
checkpoint

Common mistake

Treating landing and staging as the same mutable folder; advancing checkpoint after fetch only.

Validation

Delete staging and rebuild it from raw; accepted + quarantine must reconcile to source scope and checkpoint must remain unchanged on failure.

Changed retry

Change the target write to fail after staging; prove raw/run evidence survives and state does not advance.

A02 - DE08-02 | Defensive CSV ingestion

Concrete solved example

For the good four-row fixture, preserve raw bytes and parse 4 accepted / 0 quarantine. In a bad fixture, keep leading-zero IDs as text and classify malformed fields instead of dropping them.

Worked proof / evidence

```
assert source_rows == accepted_rows + quarantine_rows
assert all(isinstance(x, str) for x in accepted.id)
```

Common mistake

Using split(",") or implicit numeric conversion on identifiers.

Validation

Compare source row count, accepted/quarantine counts, ID text and raw SHA-256.

Changed retry

Add a quoted comma and one malformed quantity; prove one stays valid and one quarantines.

A03 - DE08-03 | JSON and nested-record ingestion

Concrete solved example

Flatten J3001 to exactly two item-grain rows keyed by (order_id,line_no), while retaining the raw order event and reconciling item count/value.

Worked proof / evidence

```
assert flattened.groupby(["order_id", "line_no"]).size().max() == 1
assert len(flattened) == 2
```

Common mistake

Flattening nested items without declaring new grain or preserving order identity.

Validation

Source item count and flattened item count must agree; item keys must be unique.

Changed retry

Add a third line item and an optional nested field; predict the new flattened count.

A04 - DE08-04 | Parquet ingestion

Concrete solved example

Inspect schema, row count, row groups and a projected read. Save evidence from real PyArrow on learner PC; simulator/static checks do not substitute.

Worked proof / evidence

```
python practice/parquet/parquet_lab.py
```

Common mistake

Calling a file Parquet because of its extension; reading all columns and never inspecting metadata.

Validation

Compare logical row count/key set to source and prove the projection contains only requested columns.

Changed retry

Project a different column subset and compare schema/row count.

A05 - DE08-05 | REST contracts/status/timeouts

Concrete solved example

Classify /health 200 as transport+contract success, unauthenticated order endpoint 401 as auth failure, missing path 404 as client/path error. JSON decoding comes after status/content-type policy.

Worked proof / evidence

```
r=requests.get(url,timeout=10)
r.raise_for_status()
assert "application/json" in r.headers.get("content-type","")
```

Common mistake

Treating any JSON body as success or omitting timeout.

Validation

Save status, content type, parse outcome and target/state decision for each case.

Changed retry

Use a 200 text/plain endpoint and classify it as contract failure.

A06 - DE08-06 | Auth, pagination and rate limits

Concrete solved example

With page_size=3, fetch all 10 unique IDs; retry the one page-2 429 according to Retry-After; do not repeatedly retry 401; never log token value.

Worked proof / evidence

```
assert fetched_count == unique_id_count == 10
```

Common mistake

Stopping at first page; printing Authorization header; retrying bad credentials.

Validation

Compare ID set at two page sizes and inspect logs for token leakage.

Changed retry

Use page_size=4 and prove the final set is identical.

A07 - DE08-07 | Retries/backoff/jitter

Concrete solved example

Retry only transient classes with a bounded budget. A one-time 500 may recover; wrong auth fails fast. Writes require idempotency before retry can be called safe.

Worked proof / evidence

```
for attempt in range(max_attempts):
    try: return call()
    except Retryable: ...
```

Common mistake

catch Exception -> retry forever; retrying a non-idempotent sink write blindly.

Validation

Record attempt count, last error/status, sleep/backoff and whether target/state changed.

Changed retry

Set budget lower than failures and prove final failure is explicit.

A08 - DE08-08 | Database extraction

Concrete solved example

Use a parameterized composite boundary such as (updated_at,id) > (:ts,:id), explicit columns and deterministic ORDER BY. After (08:15,D5002), expected records are D5003 then D5004.

Worked proof / evidence

```
SELECT id,updated_at,... FROM source
WHERE (updated_at,id) > (:ts,:id)
ORDER BY updated_at,id
```

Common mistake

Timestamp-only boundary when ties exist; SELECT *; mutable query text interpolation.

Validation

Expected ordered IDs must match; read-only connection must reject mutation.

Changed retry

Create a tie at 08:15 with a higher ID and prove composite continuation.

A09 - DE08-09 | Historical raw landing

Concrete solved example

Two historical files in one run each receive immutable object identity, original filename, size and SHA-256. Reusing a run ID with changed bytes must never overwrite the earlier object.

Worked proof / evidence

```
first_bytes = Path(obj).read_bytes()
# second changed-content collision -> fail/new identity
assert Path(obj).read_bytes() == first_bytes
```

Common mistake

Using run_id/filename as a mutable path without collision policy.

Validation

Identical replay is safe; changed-content collision preserves first bytes/hash; rapid runs receive collision-resistant IDs.

Changed retry

Call landing twice in the same second with same filename but different bytes.

A10 - DE08-10 | Incremental loads/high-water marks

Concrete solved example

Initial v1 fetch returns 10 unique records and candidate checkpoint (2026-08-05T09:20:00Z,A4010). Save that checkpoint only after durable output validation. Timestamp ties A4007/A4008 prove ID tie-break is necessary.

Worked proof / evidence

```
rows,candidate = fetch_incremental(state)
validate_and_save(rows)
commit_state(candidate)
```

Common mistake

Committing candidate state inside fetch; timestamp-only watermark.

Validation

Failure between fetch and durable save must leave old checkpoint byte-identical.

Changed retry

Inject save failure after fetch and rerun from unchanged state.

A11 - DE08-11 | CDC semantics

Concrete solved example

Apply ordered change events by event/change identity. Duplicate event_id is idempotent; current state leaves A5001 paid and A5002 removed while event history remains auditable.

Worked proof / evidence

```
assert current["A5001"].status=="paid"
assert "A5002" not in current
```

Common mistake

Treating event log as current-state table or applying duplicate events twice.

Validation

Reconcile event count/history to current-state effects and duplicate-event handling.

Changed retry

Add a later update to deleted A5002 and state your resurrection policy.

A12 - DE08-12 | Schema drift/time zones/quarantine

Concrete solved example

Land v2 raw evidence, quarantine unsupported contract rows, contribute zero v1 staging rows and keep live v1 checkpoint unchanged. Reject/flag offset-less timestamps rather than inventing local time.

Worked proof / evidence

```
assert staged_v1 == 0
assert checkpoint_after == checkpoint_before
```

Common mistake

Silently renaming/ignoring fields or assigning local timezone to naive timestamps.

Validation

Observed schema, rejection reason, raw hash and state equality are saved.

Changed retry

Add one backward-compatible optional field and decide whether it can pass.

A13 - DE08-13 | Checkpoint/state and replay

Concrete solved example

Checkpoint is owned by the durable target/validation boundary, not by the fetch loop. Replay may create new run evidence but no-commit replay leaves live state unchanged.

Worked proof / evidence

```
before=state_file.read_bytes()  
run(commit=False)  
assert state_file.read_bytes()==before
```

Common mistake

State mutation during read/fetch or quarantine write.

Validation

Byte-compare state before/after failure/no-commit path and reconcile output durability.

Changed retry

Crash after output write but before checkpoint commit; rerun and prove idempotent target behavior.

A14 - DE08-14 | Idempotent landing/content identity

Concrete solved example

SHA-256 proves byte identity, not business uniqueness. Two identical raw landings can coexist as run evidence while staging/target enforces one logical row per business key under its rule.

Worked proof / evidence

```
assert sha(run1)==sha(run2)  
assert target.business_key.is_unique
```

Common mistake

Using hash equality as business dedup rule or overwriting prior run evidence.

Validation

Preserve both run manifests and prove logical target uniqueness separately.

Changed retry

Replay identical file under new run ID and compare raw evidence vs logical target.

A15 - DE08-15 | Late data/event vs processing time

Concrete solved example

Keep event_time unchanged. Compute lateness from processing/arrival reference and declared tolerance; L2 is late at 60 minutes, and classification may change if tolerance changes.

Worked proof / evidence

```
lateness = processing_time - event_time  
is_late = lateness > tolerance
```

Common mistake

Rewriting event_time to processing_time or hiding late records.

Validation

Save event_time, processing_time, tolerance and classification; recompute after policy change.

Changed retry

Change tolerance to 120 minutes and predict classification before running.

A16 - DE08-16 | Ingestion observability

Concrete solved example

A run record needs source/accepted/quarantine counts, retries, state before/after, status and latency. SUCCESS with a missing row must be caught by reconciliation, not trusted because the log says success.

Worked proof / evidence

```
assert source_count == accepted_count + quarantine_count
```

Common mistake

Monitoring only process status/CPU and omitting data controls.

Validation

Deliberately drop one accepted row and show the equation fails despite SUCCESS status.

Changed retry

Add per-reason quarantine counts and one freshness metric.

A17 - DE08-17 | Integration/backfill proof

Concrete solved example

A strong integrated run proves immutable landing, contract validation, quarantine, bounded retry, composite checkpoint, replay safety, late-event reasoning, structured metadata and failure recovery.

Worked proof / evidence

```
evidence = [raw_hash, counts, quarantine_reasons, checkpoint_before, checkpoint_after, target_keys, failure_recovery_note]
```

Common mistake

Calling the pipeline complete because one happy-path output file exists.

Validation

Run one normal load, identical replay and injected failure; compare target/state/evidence each time.

Changed retry

Backfill a historical interval with a separate documented policy and prove live forward state does not regress.